

Rapport TP1 - Apprentissage automatique

Bastien Jallais

Octobre 2025

Contents

1	Introduction	2
2	Choix des caractéristiques	2
2.1	Choix des caractéristiques	2
2.2	Disposition mis en place pour l'implémentation des caractéristiques	4
3	Préparation des données	4
3.1	Nettoyage de base	5
3.2	Normalisation des mots	5
4	Classification et analyse comparative	5
4.1	AdaBoost	6
4.2	Bagging	9
4.3	Arbre de décisions	10
4.4	GBoost	13
4.5	Random forest	15
4.6	Analyse des points forts et points faibles des algorithmes	18
4.7	Comparaison des différents modèles	19
5	Évaluation sur les données de test	20
6	Deuxième version	20
7	Conclusion	23
8	Annexes	23

1 Introduction

Ce rapport présente le projet de classification de tweets selon leur connotation positive ou négative. Il met en lumière les choix réalisés, notamment la sélection et l'implémentation des caractéristiques ainsi que l'évaluation comparative de différents algorithmes de classification. Les résultats obtenus seront discutés afin de justifier les décisions prises et de dégager les approches les plus efficaces. Enfin, une explication plus détaillée du code est disponible directement sous forme de commentaires dans les fichiers Python.

2 Choix des caractéristiques

Ce rapport sera construit en deux versions distinctes. La première constituera la version principale, dans laquelle j'étudierai le comportement de mes modèles en fonction des six caractéristiques principales, détaillées plus bas.

Dans la deuxième version, l'objectif est d'expérimenter différentes combinaisons de caractéristiques, en en supprimant ou en ajoutant certaines. Le but est de tendre vers une sélection plus pertinente afin d'identifier le ou les modèles présentant le taux d'erreur le plus faible.

L'analyse des six caractéristiques principales retenues pour la première version sera présentée de manière détaillée, conformément aux consignes du TP. En revanche, l'évaluation des autres combinaisons de caractéristiques sera plus concise, puisqu'elle suivra la même logique de comparaison établie dans la première version. Le but de cette deuxième étape est donc d'optimiser les résultats obtenus sans alourdir inutilement la discussion.

2.1 Choix des caractéristiques

Dans cette section, je vais expliquer les raisons qui m'ont conduit, après réflexion, à retenir ces caractéristiques comme principales pour ma première version. La plupart de ces caractéristiques ont été choisies en s'inspirant des exemples présentés dans l'énoncé. L'objectif est d'obtenir, pour la plupart de ces caractéristiques, un score de positivité.

Score de positivité : Pour une caractéristique c dans un tweet, le score de positivité S_c est défini comme la différence entre le nombre d’occurrences d’éléments positifs et le nombre d’occurrences d’éléments négatifs de cette même caractéristique.

$$S_c = P_c - N_c$$

où :

- P_c est le nombre d’occurrences d’éléments positifs pour la caractéristique c ,
- N_c est le nombre d’occurrences d’éléments négatifs pour la caractéristique c .

Nom de la feature	Type	Description / Rôle
score_mots_pos_neg	Entier	Score net des mots positifs et négatifs dans le tweet : +1 pour chaque mot positif, -1 pour chaque mot négatif. Indique le sentiment global.
score_emojis_pos_neg	Entier	Score net des emojis positifs et négatifs : +1 pour chaque emoji positif, -1 pour chaque emoji négatif. Capture l’expression émotionnelle directe.
nb_exclamations_intensifieurs	Entier	Somme du nombre de points d’exclamation et des mots amplificateurs (ex. : "very", "so"). Permet de mesurer l’intensité émotionnelle que l’utilisateur à voulu faire passer.
nb_negations	Entier	Nombre de mots de négation (ex. : "not", "no"). Très utile pour comprendre le sentiment réel d’une phrase ayant un sens inversé.
nb_majuscules	Entier	Nombre de mots entièrement en majuscules. Les majuscules indiquent souvent un sentiment fort comme l’excitation ou la colère.
nb_moycaracteres	Réel	Longueur moyenne des mots dans le tweet (après nettoyage de la ponctuation). Permet de détecter le style (informel / sérieux) il peut également s’avérer utile en combinaison avec d’autres caractéristiques.

Table 1: Présentation des 6 caractéristiques principales utilisées pour l’analyse de sentiment des tweets.

2.2 Disposition mis en place pour l'implémentation des caractéristiques

Ces caractéristiques ont été choisies car elles sont à la fois simples à extraire et suffisamment expressives pour différencier les tweets à connotation positive et négative.

L'implémentation repose sur un fichier de lexiques contenant différents ensembles de mots et d'emojis classés par catégorie (positifs, négatifs, intensifieurs, négations, etc.). Chaque tweet est ensuite traité pour vérifier la présence de ces éléments et calculer les scores associés. Concrètement :

- Chaque mot et chaque emoji est nettoyé (suppression de la ponctuation, mise en minuscule) avant d'être comparé aux lexiques.
- Le score *positif/négatif* est obtenu en incrémentant ou décrémentant un compteur.
- Les emojis, les intensifieurs, les négations et les exclamations sont comptabilisés via des fonctions dédiées.
- La longueur moyenne des mots est calculée en divisant le nombre total de caractères par le nombre de mots.
- Le nombre total de mots est mesuré pour chaque tweet et stocké tel quel.

3 Préparation des données

Les données provenant de texte brut issu de tweets ont de fortes chances d'être bruitées et informelles en raison de la présence d'abréviations, d'expressions et de ponctuations incohérentes, etc. Pour réduire l'incohérence et le bruit des données, certains prétraitements seront effectués avant que les données ne soient fournies au modèle et entraînées avec les différentes caractéristiques présentées plus haut. C'est également à cette étape que je décide d'enrichir le fichier *lexique.txt* notamment pour les mots positifs, négatifs, et intensifieurs. Ce fichier est enrichi à partir de données trouvées dans des dictionnaires disponibles sur Internet (voir annexe pour la source). Le prétraitement des données est implémenté dans le fichier *pre_traitement_donnees.py*. Des explications détaillées concernant l'implémentation en Python sont directement disponibles dans les commentaires du fichier.

3.1 Nettoyage de base

Les tweets contiennent souvent des éléments qui n'apportent que peu de valeur à l'analyse des sentiments, tels que les URL, les hashtags ou les mentions. Comme les caractéristiques retenues dans mon étude ne reposent pas sur ces éléments, la première étape de nettoyage consiste à les traiter ou les supprimer. Concrètement, les hashtags sont normalisés en conservant uniquement le mot (`#happy` devient `happy`), tandis que les URL et les mentions sont retirées du texte. Cela permet de réduire le bruit tout en préservant l'information utile.

3.2 Normalisation des mots

La normalisation du texte est réalisée en deux étapes complémentaires. Tout d'abord, le lexique est enrichi par la standardisation d'abréviations et de termes familiers. Par exemple, `lol` est remplacé par `laughing` et `omg` par `oh my god`, afin de mieux capter le sentiment exprimé. Ensuite, les répétitions de lettres exagérées sont réduites (`soooo good` devient `so good`). Bien que ce type de répétition puisse être un indice d'intensité émotionnelle, j'ai choisi de ne pas l'intégrer afin de ne pas biaiser les autres caractéristiques retenues (par exemple, le comptage des mots positifs ou négatifs).

4 Classification et analyse comparative

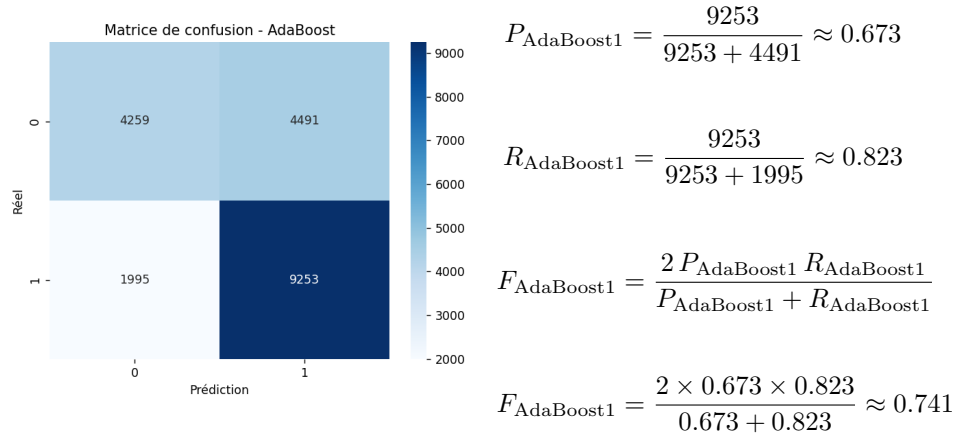
Une fois la normalisation des mots effectuée, je crée le fichier `training_data.csv` où chaque ligne est un tweet, et chaque colonne une caractéristique. C'est à partir de ce fichier que je sépare les données en données d'entraînement et en données de validation, avec une répartition respective de 80% et 20%.

Pour chaque algorithme, deux fonctions principales ont été implémentées : `train` et `evaluate`. La fonction `train` prend en paramètre les données d'entraînement et a pour rôle d'entraîner le modèle. Dans un premier temps, des valeurs de paramètres arbitraires sont utilisées afin d'obtenir une précision de base du modèle. Par la suite, un second modèle est construit en utilisant `GridSearchCV`, un outil de la bibliothèque `scikit-learn` qui permet de rechercher automatiquement les combinaisons d'hyperparamètres offrant la meilleure précision. Les modèles avec les paramètres de valeur arbitraire seront enregistrés dans "models/base" et les modèles optimisés dans "models/opt_results". Ainsi, lors de l'exécution du projet, il est possible de tester et de comparer ces deux approches d'entraînement : l'une simple et l'autre optimisée.

La fonction `evaluate` prend quant à elle en paramètre les données de validation et permet d'évaluer les performances du modèle. Elle calcule l'exactitude (accuracy) et génère une matrice de confusion, à partir de laquelle il est possible de déterminer le rappel (recall), la F-mesure (F1-score) et la précision.

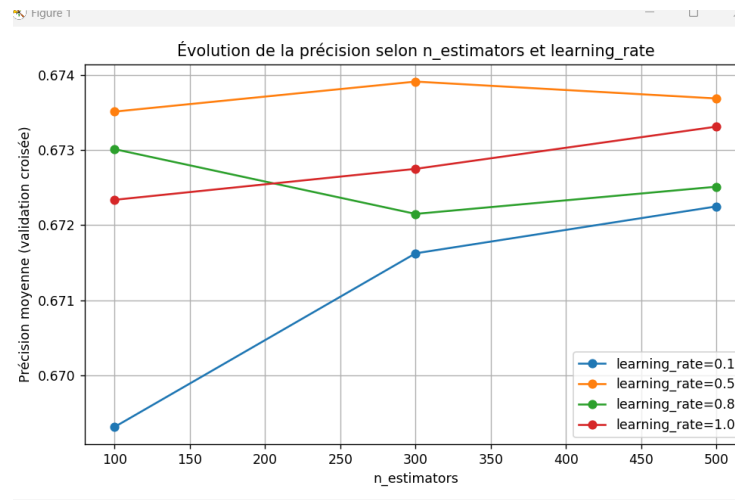
4.1 AdaBoost

AdaBoost avec hyperparamètres de base :

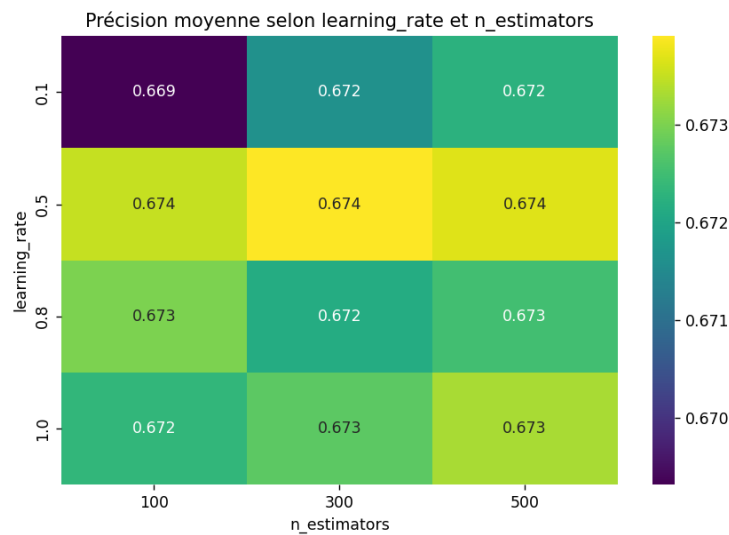


L'accuracy globale obtenue est de $acc = 0.6757$. Ces premiers résultats montrent qu'AdaBoost parvient à bien identifier la classe positive (rappel élevé de 0.823), cependant il identifie un nombre important de faux positifs (précision limitée à 0.673).

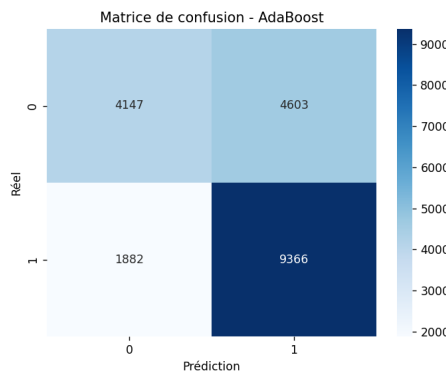
AdaBoost avec GridSearchCV (hyperparamètre optimisé):



Les valeurs de précision tournent entre 0.672 et 0.674, on peut donc comprendre que le modèle reste très simple peu importe les valeurs du couple (learning_rate, n_estimators). Ce suggère que ce modèle est peu sensible aux variations de ces deux hyperparamètres.



Le meilleur learning rate semble néanmoins être 0.5, avec une précision constante de 0.674 en son maximum. Un learning faible (0.1) commence plus bas mais s'améliore en fonction du nombre d'estimateurs et un learning rate élevé de (0.8 ou 1) ne dégrade pas les performances. On observe également qu'augmenter le nombre d'estimateurs améliore la performances (surtout pour un learning rate faible), cependant cette amélioration reste faible est marginal ce qui indique toujours une certaine robustesse du modèle.



$$P_{\text{AdaBoost2}} = \frac{9366}{9366 + 4603} \approx 0.671$$

$$R_{\text{AdaBoost2}} = \frac{9366}{9366 + 1882} \approx 0.833$$

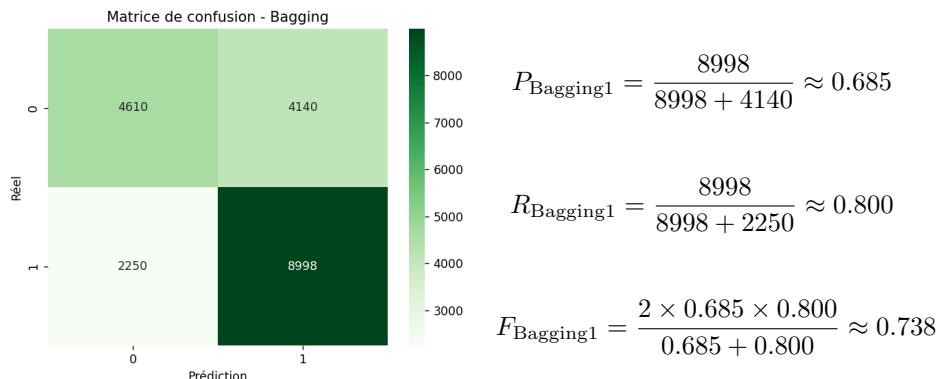
$$F_{\text{AdaBoost2}} = \frac{2 P_{\text{AdaBoost2}} R_{\text{AdaBoost2}}}{P_{\text{AdaBoost2}} + R_{\text{AdaBoost2}}}$$

$$F_{\text{AdaBoost2}} = \frac{2 \times 0.671 \times 0.833}{0.671 + 0.833} \approx 0.743$$

L'accuracy finale demeure identique ($acc = 0.6757$), confirmant que l'optimisation n'apporte qu'un bénéfice très limité. Si le rappel progresse légèrement, la précision diminue légèrement, ce qui entraîne une amélioration marginale du score $F1$. AdaBoost semble donc plafonner autour de 67% d'accuracy

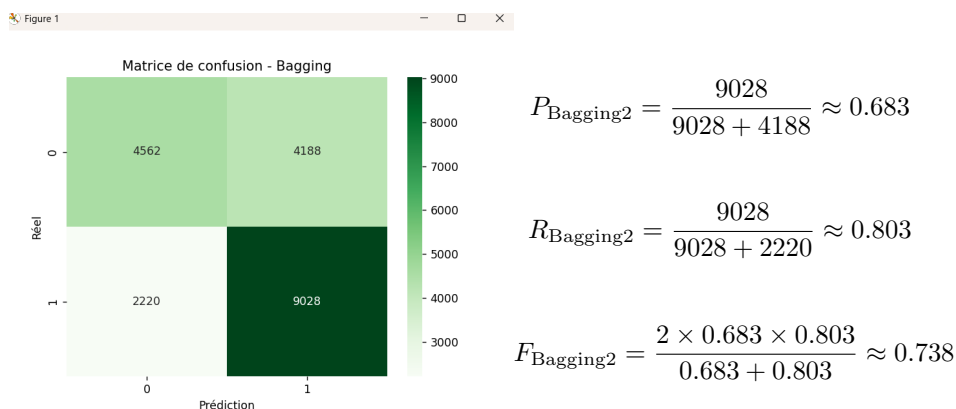
4.2 Bagging

Bagging avec hyperparamètres par défaut



L'accuracy obtenue est $acc = 0.6805$. Ce premier résultat met en évidence une meilleure précision que celle observée sur AdaBoost en configuration par défaut. Le modèle est capable de détecter efficacement la classe positive (rappel de 0.800), mais il génère encore un volume significatif de faux positifs, comme l'indique une précision modérée (0.685).

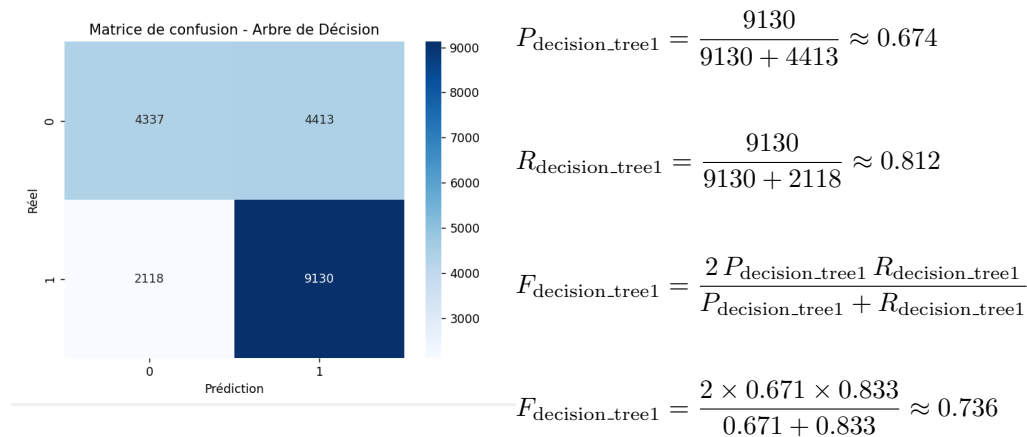
Bagging optimisé via GridSearchCV



L'accuracy finale est très proche de la version initiale ($acc = 0.679$). L'optimisation n'apporte donc pas de gain significatif, suggérant que Bagging atteint rapidement ses performances maximales avec les paramètres par défaut. D'un point de vue comparatif, Bagging présente de meilleures performances globales qu'AdaBoost sur l'accuracy et la précision. Cependant, contrairement à AdaBoost, son rappel varie peu après optimisation.

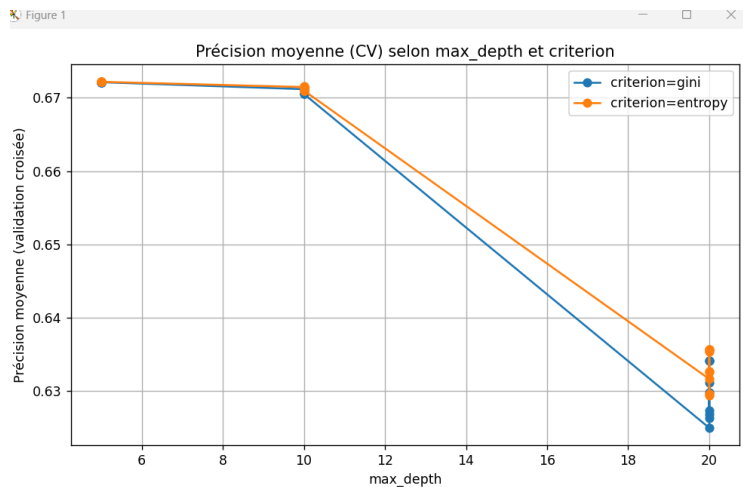
4.3 Arbre de décisions

Arbre de décisions avec hyperparamètres de base :

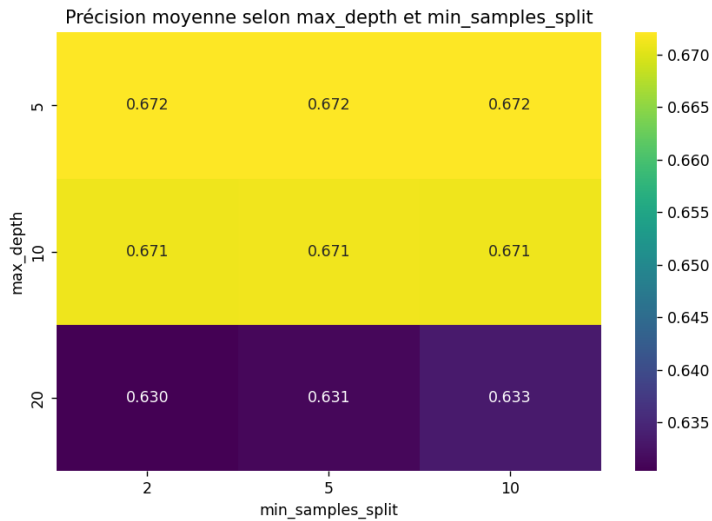


L'exactitude ou accuracy (acc) de ce modèle est $\text{acc} = 0.6734$. On observe donc, comme d'habitude, une bonne capacité à détecter la classe positive (rappel de 0,812). Cependant, il y a toujours la même tendance à générer un gros volume de faux positifs, comme on peut le voir avec la précision moyenne de 0,674.

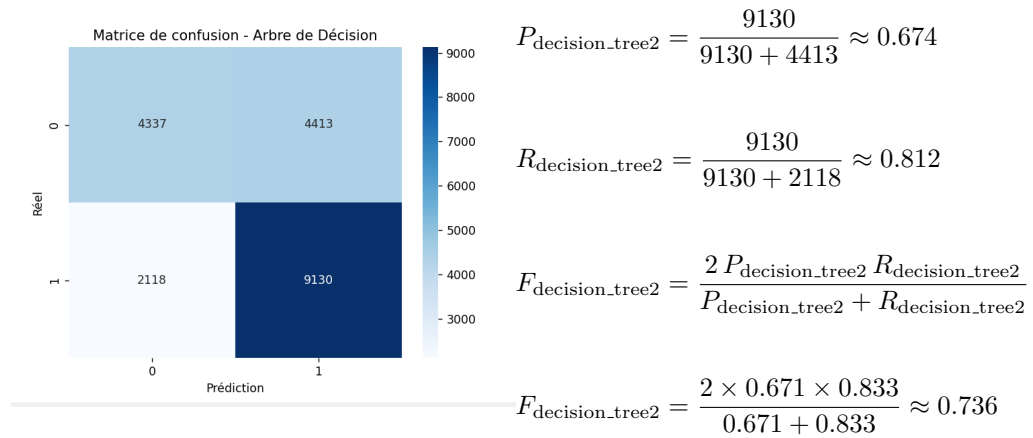
Arbre de décision avec GridSearchCV (hyperparamètre optimisé):



On remarque que les meilleures performances se trouvent autour de $\text{max_depth} = 5$. Lorsque la profondeur augmente à 10 puis 20 la précision diminue progressivement, cela peut être dû au fait que l'arbre se généralise mieux lorsqu'il est peu profond car il évite le problème de l'overfitting. On voit également que l'entropie est légèrement meilleur que le gini bien que l'écart reste faible.



Comme dit précédemment, les meilleures performances (≈ 0.672) sont obtenues lorsque la profondeur de l'arbre est de 5, peu importe le nombre `min_sample_split`. On voit très clairement cela en remarquant que la précision chute significativement (≈ 0.630 à 0.633) lorsque la profondeur de l'arbre est de 20. On peut donc conclure que la profondeur est dans ce cas le paramètre le plus influent sur la performance, et que le modèle reste robuste aux variations de `min_sample_split`.

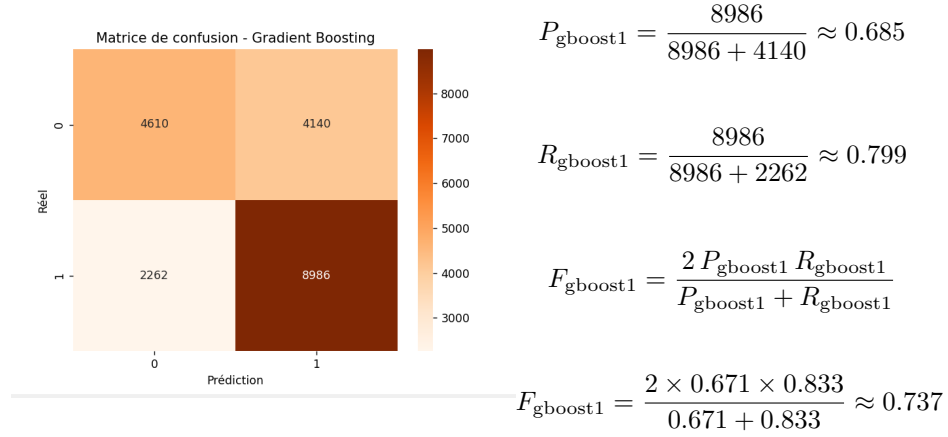


L'exactitude ou accuracy (`acc`) de ce modèle est $\text{acc} = 0.6734$, soit exactement la même que pour le modèle non optimisé. Cela s'explique simplement par le fait que j'avais arbitrairement choisi les meilleurs paramètres lors de l'entraînement de mon premier modèle.

L'essentiel ici ne se situe pas au niveau de cette similarité, mais au niveau des résultats quasi identiques pour la précision et le rappel sur les algorithmes AdaBoost, Bagging et Decision tree. Soit des résultats montrant que les modèles ont tendance à bien classer les positifs mais également à classer une grande quantité de faux positifs. Cela suggère que ces modèles manquent de capacité à discerner correctement la classe négative et qu'une amélioration des caractéristiques peut être nécessaire pour améliorer leurs performances.

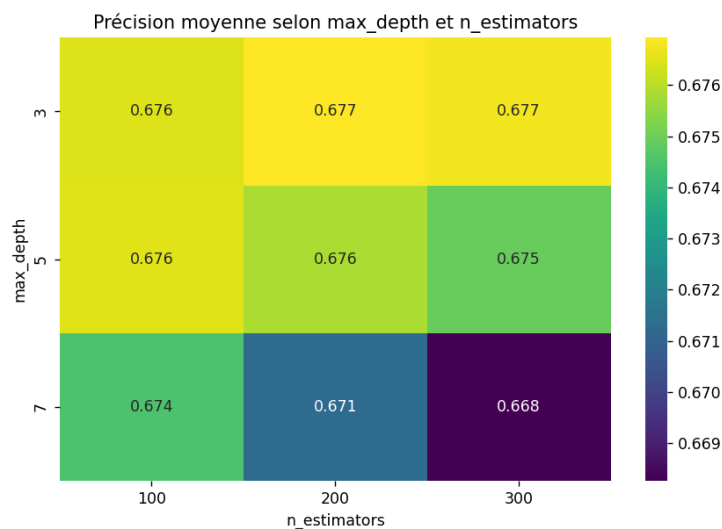
4.4 GBoost

GBoost avec hyperparamètres de base :

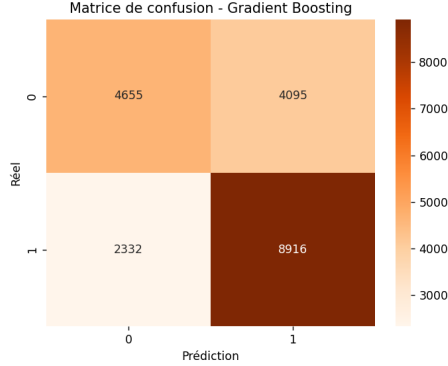


L'exactitude ou accuracy (acc) de ce modèle est $\text{acc} = 0,6799$. La matrice de confusion montre que GBoost avec des hyperparamètres de base présente de bonnes performances globales. Comme pour les autres modèles, il présente une tendance à confondre certains tweets négatifs avec des tweets positifs.

GBoost avec GridSearchCV (hyperparamètre optimisé):



Le graphique montre que Gboost atteint ses meilleures performances avec une profondeur que l'on peut qualifier d'intermédiaire (3 à 5), et un nombre d'estimateurs situé autour de 100 à 200. Comme pour les modèles d'arbre de décision, on peut comprendre qu'une trop grande profondeur diminue les performances à cause du surapprentissage, le modèle devient trop complexe pour généraliser correctement les données de validation. Néanmoins, ce graphique montre également que les scores d'accuracy varient très peu, entre 0.668 et 0.677, ce qui indique un modèle relativement stable aux changements des hyperparamètres.



$$P_{\text{gboost2}} = \frac{8916}{8916 + 4095} \approx 0.685$$

$$R_{\text{gboost2}} = \frac{8916}{8916 + 2332} \approx 0.793$$

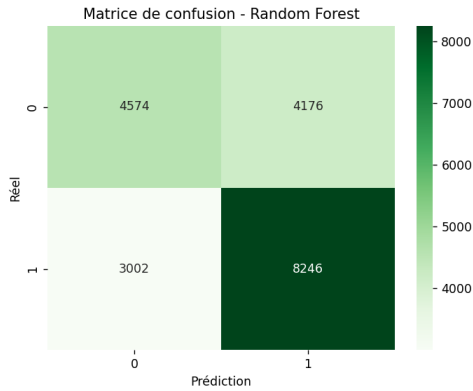
$$F_{\text{gboost2}} = \frac{2 P_{\text{gboost2}} R_{\text{gboost2}}}{P_{\text{gboost2}} + R_{\text{gboost2}}}$$

$$F_{\text{gboost2}} = \frac{2 \times 0.671 \times 0.833}{0.671 + 0.833} \approx 0.735$$

L'exactitude ou accuracy (acc) de ce modèle est $\text{acc} = 0,6799$. La comparaison entre le modèle de base ainsi que le modèle optimisé montre que l'optimisation n'a pas permis une amélioration notable des performances, les métriques restent extrêmement proches. On constate même une légère baisse du rappel et du F1-score. Comme pour AdaBoost, cela peut s'expliquer par un choix arbitraire déjà rigoureux lors de la construction du modèle sans utilisation de GridSearchCV.

4.5 Random forest

Random forest avec hyperparamètres de base :



$$P_{\text{random_forest1}} = \frac{8246}{8246 + 4176} \approx 0.685$$

$$R_{\text{random_forest1}} = \frac{8246}{8246 + 3002} \approx 0.799$$

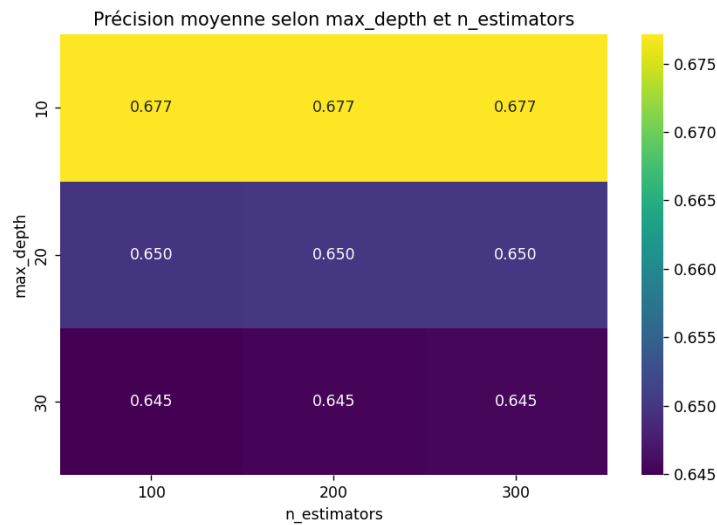
$$F_{\text{random_forest1}} = \frac{2 P_{\text{random_forest1}} R_{\text{random_forest1}}}{P_{\text{random_forest1}} + R_{\text{random_forest1}}}$$

$$F_{\text{random_forest1}} = \frac{2 \times 0.671 \times 0.833}{0.671 + 0.833} \approx 0.737$$

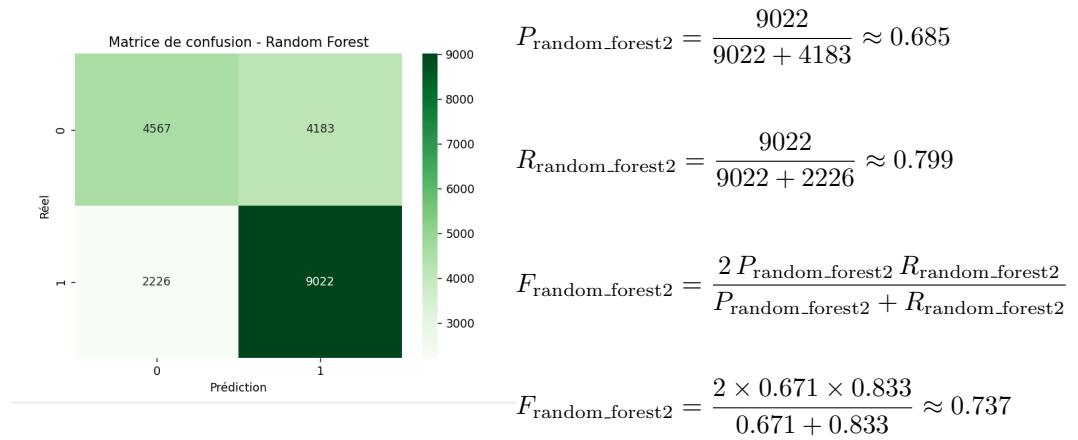
Malgré un bon rappel, Random Forest avec des hyperparamètres de base obtient une précision bien en deçà des autres modèles (0.641). Cependant son F1-score reste correct, mais il n'est pas assez élevé pour se démarquer par rap-

port aux autres modèles.

Random forest avec GridSearchCV (hyperparamètre optimisé):



Comme pour les arbres de décision, le paramètre max_depth servant à gérer la profondeur de l'arbre reste le plus important. Lorsque la profondeur augmente à 20, l'accuracy chute à environ 0.650 et diminue encore légèrement lorsque celle-ci passe à 30 (0.645). En revanche, avec une profondeur plus faible à 10, on obtient de meilleures performances avec une accuracy allant jusqu'à 0.677. Le nombre d'arbres a quant à lui un impact limité et son augmentation n'apporte pas de gain significatif.



L'optimisation avec GridSearchCV permet d'améliorer l'accuracy du modèle (0.6795), mais elle n'entraîne pas de progression notable sur les autres métriques. Il ne parvient toujours pas à rivaliser avec les modèles les plus performants (notamment AdaBoost2). On remarque que random forest souffre comme les autres du même problème qui est de générer un grand nombre de faux positifs.

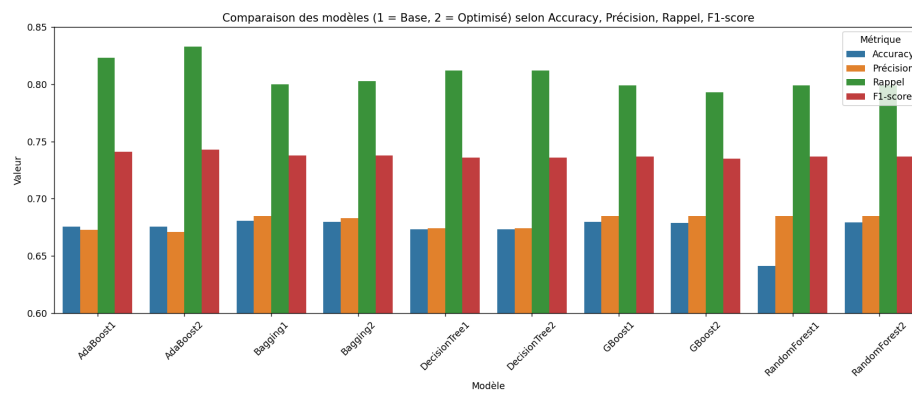
4.6 Analyse des points forts et points faibles des algorithmes

Modèle	Points forts	Points faibles
AdaBoost	• Très bon rappel (capte efficacement la classe positive). • Léger gain de F1-score après optimisation. • Modèle relativement stable, peu sensible aux hyperparamètres.	• Précision limitée (beaucoup de faux positifs). • Accuracy plafonnant autour de 0.675. • Gain marginal après GridSearchCV.
Bagging	• Meilleure accuracy globale (0.6805). • Bon compromis précision/rappel. • Performant dès les paramètres par défaut (robuste).	• Génère encore un volume notable de faux positifs. • Optimisation n'apporte pas de gain significatif. • Légèrement moins performant qu'AdaBoost en rappel.
Decision Tree	• Modèle très interprétable. • Bon rappel (capacité à identifier la classe positive). • Bon équilibre de profondeur autour de <code>max_depth = 5</code>.	• Sensible au surapprentissage si mal réglé. • Performances globales moyennes (F1 inférieur aux ensembles). • Peu compétitif par rapport aux méthodes ensemblistes.
GBoost	• Bon équilibre entre précision et rappel. • Modèle stable aux variations d'hyperparamètres. • Accuracy compétitive (0.6799).	• Améliorations limitées après optimisation. • Légère baisse du F1-score après GridSearchCV. • Tendance à générer des faux positifs.
Random Forest	• Bon rappel (capacité à identifier la classe positive). • F1-score correct grâce au rappel. • Légère amélioration de l'accuracy après optimisation.	• Accuracy de base nettement inférieure (0.641). • Même optimisé, reste moins performant que les modèles boosting/bagging. • Produit trop de faux positifs sans avantage distinct.

Table 2: Synthèse des points forts et points faibles des modèles testés.

D'un point de vue global, les modèles semblent tous souffrir d'un même problème, ils classent correctement la majorité des tweets positifs mais ont tendance à classer une grande quantité de tweets négatifs comme positif. Ce problème étant commun à notre ensemble de modèles, il semblerait que la solution pour le résoudre soit de revoir et d'améliorer les caractéristiques utilisées. C'est ce qu'on tentera de faire dans la deuxième version du projet.

4.7 Comparaison des différents modèles



Modèle	Accuracy	Précision	Rappel	F1-score
AdaBoost1	0.675668	0.673	0.823	0.741
AdaBoost2	0.675718	0.671	0.833	0.743
Bagging1	0.680468	0.685	0.800	0.738
Bagging2	0.679568	0.683	0.803	0.738
DecisionTree1	0.673417	0.674	0.812	0.736
DecisionTree2	0.673417	0.674	0.812	0.736
GBoost1	0.679868	0.685	0.799	0.737
GBoost2	0.678618	0.685	0.793	0.735
RandomForest1	0.641064	0.685	0.799	0.737
RandomForest2	0.679518	0.685	0.799	0.737

Table 3: Comparaison des performances des modèles (Base vs Optimisé).

Voici les résultats qu'on obtient, la question est maintenant de savoir quelle métrique prioriser afin de choisir le modèle qui semble généraliser au mieux les données. L'exactitude (accuracy) semble être dans ce cas un bon indicateur, mais pas suffisant seul, la précision ainsi que le rappel sont importants mais moins prioritaires seuls également. Le F1-score semble être la meilleure métrique, elle permet de montrer la capacité du modèle à bien classer les tweets positifs et négatifs, et donc de ne pas sacrifier une métrique par rapport à une autre.

En partant de ce principe, on remarque que c'est le Bagging de base qui obtient la meilleure accuracy, le modèle AdaBoost optimisé (AdaBoost2) pour sa part obtient le meilleur équilibre en surpassant les autres modèles au niveau du F1-score ainsi qu'au niveau du rappel. Cela montre que c'est celui qui est

le plus capable de capturer efficacement la majorité des sentiments d'un tweet sans sacrifier la précision. Ce sera donc le modèle final qui sera sélectionné pour l'évaluation sur les données de test.

5 Évaluation sur les données de test

Afin d'évaluer mon modèle, 20 tweets positifs et 20 tweets négatifs ont été choisis au hasard parmi les cent premiers tweets classés dans le fichier "test_prediction.csv". Ensuite ceux-ci sont classés dans un fichier "20pos_20neg.csv" (disponible à la racine du projet) avec l'ajout d'une colonne "HumanSentiment" permettant d'évaluer selon mon propre ressenti la classification d'un tweet ; cela afin de mieux comprendre les points forts et points faibles du modèle. Voici après analyse ce qu'on peut retenir :

- Le modèle peut confondre intensité d'expression ou présence d'exclamation (de sentiment fort) avec un sentiment purement positif.
- Lorsque le modèle ne sait pas ou n'identifie rien d'associé à la tristesse pure, il semble automatiquement supposer un sentiment positif par défaut.
- Il ne gère pas correctement les émotions ambiguës ou composées, comme par exemple "great, but need a scolding".

Les deux premiers points expliquent la grande présence de faux positifs, également on peut expliquer cela par le fait qu'il y a plus de caractéristiques pour identifier un sentiment positif (nombre de mots positifs ainsi que nombre de points d'exclamation par exemple). Pour corriger cela on peut améliorer la qualité des caractéristiques et/ou rendre le modèle plus compréhensible à la sémantique de mots associés à du négatif comme *hospital*.

Pour le troisième point il ne semble pas réellement y avoir de solution possible si on reste sur un modèle classique de classification binaire. La solution serait soit de passer à un modèle de classification multi-label (joie, nostalgie, anxiété, tristesse, fierté, frustration, ...) ou au minimum un modèle plus complet comme un modèle de classification probabiliste allant de 1 très positif à 0 très négatif ou une classification également multi-label mais plus simple (positif, négatif, mixte, neutre).

6 Deuxième version

Comme dit dans l'introduction, cette version a pour but d'améliorer l'efficacité globale et plus particulièrement la précision des modèles, sans pour autant alourdir la discussion. Pour cela, on détaillera ici les différentes caractéristiques mises en place ainsi que les prétraitements effectués sur les données. Puis, on étudiera, comme pour la première version, à l'aide de graphiques de comparaison, les résultats des différents modèles au niveau de la précision, de l'exactitude, du rappel ainsi que de la f_mesure. Ces différents calculs ainsi que la recherche des

meilleurs hyperparamètres seront effectués en suivant le même procédé que la première version, c'est pourquoi on ne s'attardera pas dessus ici.

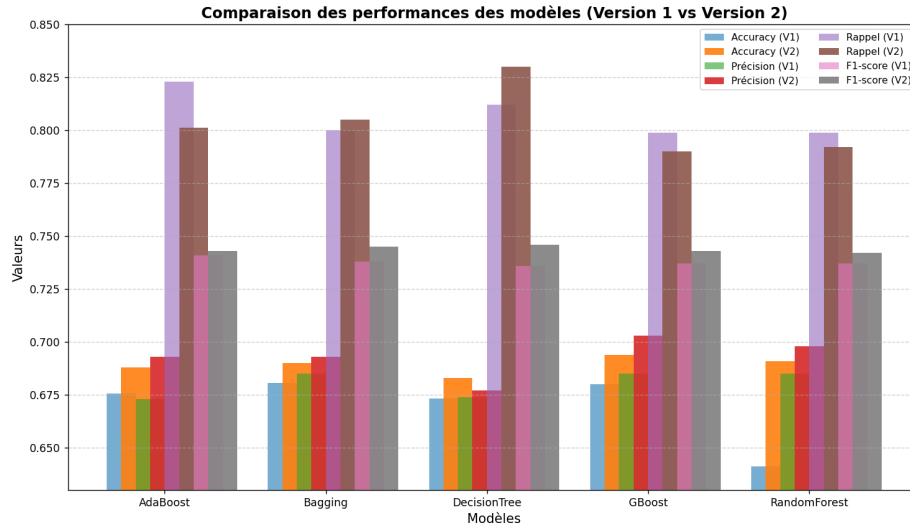
Voici les différentes caractéristiques ajoutées dans le but d'améliorer la qualité de mes modèles. On note ici que pour le bon fonctionnement de certains de ces nouveaux caractéristiques, aucun prétraitement ne sera effectué :

- **n_urls** : compte le nombre d'URL dans un tweet, les URL sont souvent promotionnelles / informatives ou peuvent servir à amplifier un jugement.
- **n_mentions** : les mentions sont souvent en corrélation avec des interactions. Sur X, les replies sont souvent agressives et donc négatives (le but est donc ici de diminuer le nombre de faux positifs).
- **n_hashtags** : leur présence peut indiquer une forme d'intensité dans le tweet.
- **n_hashtag_pos / n_hashtag_neg** : permet, en combinaison avec le lexique, de comprendre le sentiment d'un hashtag.
- **n_elong / max_elong_run** : nombre des allongements de lettres ("soooo", "baaaad"). Cela permet de mieux capturer l'intensité émotionnelle.
- **n_question** : compte le nombre de points d'interrogation. Les questions peuvent souvent être synonymes d'incertitude ou de plainte et peuvent donc permettre de mieux comprendre les tweets négatifs.
- **n_ellipsis** : compte le nombre de points de suspension. Les points de suspension traduisent souvent une hésitation ou de la déception.
- **n_punct_runs** : compte le nombre de répétitions de ponctuation. Cela permet de mieux comprendre l'intensité d'un tweet (joie ou colère forte).
- **n_allcaps_words** : compte le nombre de mots entièrement en majuscules, encore une fois pour capter l'intensité émotionnelle ainsi qu'un sentiment fort.
- **pos_after_intensifier / neg_after_intensifier** : nombre de mots positifs / négatifs apparaissant juste après un intensifieur ("so great", "very bad"), évalue mieux la force du sentiment que si l'on fait simplement le ratio mots positifs / négatifs comme dans la première version.
- **pos_near_negation / neg_near_negation** : compte le nombre de mots positifs ou négatifs juste après une négation ("not good", "not bad"), tentative de corriger les erreurs de compréhension du sens.

Modèle	Accuracy	Précision	Rappel	F1-score
AdaBoost1	0.688	0.692	0.803	0.743
AdaBoost2	0.688	0.693	0.8011	0.743
Bagging1	0.689	0.693	0.805	0.745
Bagging2	0.690	0.693	0.805	0.745
DecisionTree1	0.683	0.677	0.830	0.746
DecisionTree2	0.683	0.677	0.830	0.746
GBoost1	0.692	0.701	0.790	0.743
GBoost2	0.694	0.703	0.790	0.743
RandomForest1	0.665	0.692	0.727	0.710
RandomForest2	0.691	0.698	0.792	0.742

Table 4: Comparaison des performances des modèles version 2(Base vs Optimisé).

À la différence de la première version, ce n'est pas le modèle Adaboost qui domine ici. Gboost obtient la meilleure exactitude (accuracy), et les algorithmes d'arbre de décision obtiennent le meilleur score au niveau du rappel et du f1-score.



On remarque qu'avec cette deuxième version, tous les modèles gagnent légèrement en accuracy, précision et F1-score. L'optimisation semble donc avoir été efficace, néanmoins, les gains ne sont pas très importants. Pour certains modèles, notamment AdaBoost ainsi que DecisionTree, le rappel a un peu diminué, mais cela s'accompagne d'une meilleure précision, ce qui veut dire que le modèle

gène moins de faux positifs. On a donc réussi à en partie corriger ce problème commun à tous les modèles de la première partie.

7 Conclusion

L'objectif de ce projet était de concevoir et d'évaluer différents modèles sur un problème de classification d'un tweet (positif ou négatif) à partir d'un ensemble de caractéristiques.

Dans la première version, six caractéristiques principales ont été retenues. Cette approche a permis d'obtenir des résultats corrects, notamment avec les modèles AdaBoost et Bagging, qui se distinguaient par un bon équilibre entre rappel et précision. Néanmoins, un problème commun à l'ensemble des modèles a été observé : une tendance à générer un grand nombre de faux positifs, traduisant une difficulté à distinguer correctement les tweets négatifs.

La seconde version du projet visait à corriger cette faiblesse en enrichissant les caractéristiques d'entrée. L'ajout de nouvelles variables, telles que le nombre d'URL, de mentions, d'allongements de lettres ou encore la gestion des négations et des intensifieurs, a permis d'améliorer légèrement les performances globales. Les scores d'accuracy, de précision et de F1-score progressent globalement. Dans cette configuration, GBoost s'impose comme le modèle offrant la meilleure exactitude, tandis que les arbres de décision atteignent les meilleurs résultats en rappel et F1-score.

Bien que les gains de performance restent modestes, ils confirment l'intérêt de l'ingénierie de caractéristiques dans ce type de tâche. Une amélioration notable du compromis précision/rappel montre que le modèle a appris à mieux limiter les faux positifs, sans sacrifier sa capacité à reconnaître les tweets positifs.

Cette deuxième version montre également les limites d'une classification binaire. Une perspective d'amélioration serait donc de passer à une classification multilabel ou probabiliste afin de mieux comprendre toutes les nuances d'émotions possibles dans un tweet.

En conclusion, ce projet a permis de démontrer les limites mais aussi la robustesse des approches classiques d'apprentissage supervisé appliquées à la classification de sentiments, tout en mettant en évidence l'importance cruciale du choix et de la qualité des caractéristiques dans les performances finales.

8 Annexes

Enrichissement du lexique - mots positifs : <https://ptrckpry.com/course/ssd/data/positive-words.txt>

Enrichissement du lexique - mots négatifs : <https://gist.github.com/mkulakowski2/4289441>

Enrichissement du lexique - négations : <https://dictionary.cambridge.org/us/grammar/british-grammar/negation.2>

Enrichissement du lexique - intensifieurs : <https://www.englishcentral.com/blog/en/using-intensifiers-in-english/>